

Video Object Segmentation and Motion Tracking

Vanshika Kumawat (M22MA207)

April 12, 2025

Abstract

This project explores **video object segmentation** and **motion tracking** using classical Computer Vision techniques. Two distinct approaches are implemented: (1) edge detection and ORB feature tracking, and (2) frame differencing and thresholding. The system performs real-time motion detection and segmentation in video streams, and is deployed using Streamlit for interactive use.

Contents

1	Introduction	3
2	Objectives	4
3	Approach 1: Edge Detection + ORB Feature Tracking	8
3.1	Pipeline	8
3.2	Frame Preprocessing and Feature Detection	8
3.3	ORB Keypoints Detection	9
3.4	Optical Flow for Segmentation	10
3.5	Lucas-Kanade Feature Tracking	10
3.6	Kalman Filter Output	11
3.7	Homography Transformation	11
4	Approach 2: Frame Differencing + Thresholding	11
4.1	Pipeline	11
4.2	Frame Difference Image	12
4.3	Motion Highlight via Thresholding	12
5	Comparison Table	13
6	Implementation Details	13
7	Results	13
8	Deployment	13

9	Screenshots of Output	14
10	Conclusion	15

1 Introduction

Video object segmentation and motion tracking are foundational techniques in the field of Computer Vision, enabling the identification and localization of moving entities within a sequence of video frames. These techniques find broad applications across diverse domains such as intelligent surveillance, traffic monitoring, autonomous navigation, and human activity analysis.

This project focuses on exploring and implementing two classical, non-deep learning approaches to effectively track and segment motion in videos. By avoiding complex neural network architectures, we aim to develop lightweight, interpretable, and real-time solutions that are computationally efficient and suitable for deployment in constrained environments.

The two approaches studied in this project are:

- **Approach 1: Edge detection combined with ORB-based feature tracking.** This technique identifies object boundaries using Canny edge detection and follows key visual features over time using the ORB (Oriented FAST and Rotated BRIEF) algorithm.
- **Approach 2: Frame differencing followed by thresholding and morphological filtering.** This method detects motion by measuring pixel-level changes between consecutive frames, isolating regions of movement using simple but effective operations.

These methods are implemented in Python using OpenCV and deployed through a user-friendly Streamlit interface, allowing users to upload videos, run the segmentation/tracking pipelines, and view the results in real-time.

2 Objectives

The core objectives of the project are as follows:

- **Motion Segmentation:** Accurately identify and isolate moving objects in video sequences using traditional Computer Vision techniques.
- **Dual Approach Implementation:** Design and implement two distinct motion tracking pipelines—one based on edge and feature tracking, and the other on pixel-wise frame differencing and thresholding.
- **Visual Output Generation:** Generate annotated video outputs that overlay motion segmentation and tracking results on top of the original footage for better visualization and analysis.
- **Interactive Deployment:** Build and deploy the complete system using Streamlit to facilitate real-time interaction. The interface allows users to upload video files, process them, and instantly observe the tracking results within the web app.
- **Performance Analysis:** Compare the performance of both approaches in terms of segmentation quality, tracking consistency, processing speed, and applicability to different video scenarios.

3. Explanation of Key Concepts

1. Canny Edge Detection

Canny edge detection is a multi-stage algorithm for identifying edges in an image. It involves the following steps:

- **Gaussian Blur** is applied to the image to reduce noise and smooth out variations.
- **Sobel operator** is used to compute the gradient magnitude and direction at each pixel.
- **Non-maximum suppression** thins out the edges by retaining only the local maxima of gradient magnitude.
- **Double thresholding** classifies pixels as strong, weak, or non-edges.
- **Edge tracking by hysteresis** connects weak edges to strong ones if they are likely to form a continuous edge.

The gradient magnitude and orientation are computed as:

$$G = \sqrt{G_x^2 + G_y^2}, \quad \theta = \tan^{-1} \left(\frac{G_y}{G_x} \right)$$

2. ORB (Oriented FAST and Rotated BRIEF)

ORB is a fast and efficient algorithm for detecting and describing keypoints in an image. It is particularly well-suited for real-time applications. It consists of:

- **FAST (Features from Accelerated Segment Test)** is used to detect corner-like regions in the image.
- **BRIEF (Binary Robust Independent Elementary Features)** creates a compact binary descriptor based on pixel intensity comparisons.
- **Orientation component** is added to handle image rotation. ORB assigns an orientation to each keypoint using intensity centroid, which makes it invariant to in-plane rotations.

ORB improves upon BRIEF by ensuring rotational invariance and uses a learning method to select the most uncorrelated binary features, making it robust and suitable for tracking objects between video frames.

3. Optical Flow

Optical flow estimates the motion of objects between consecutive frames based on intensity changes. The goal is to find the motion vector $\vec{v} = (v_x, v_y)$ such that:

$$I(x, y, t) = I(x + v_x, y + v_y, t + 1)$$

Common algorithms include:

- **Lucas-Kanade:** Assumes local constancy of flow and solves using a least-squares approach.
- **Farneback:** Estimates dense flow using polynomial expansion of neighborhoods.

4. Kalman Filter for Tracking

Kalman Filter is a recursive algorithm used to estimate the state of a dynamic system from a series of noisy measurements. It consists of two steps:

- **Prediction:** Uses the system's dynamics to predict the next state.
- **Correction:** Updates the prediction based on the new measurement.

It is effective for tracking objects in video sequences, as it adapts to motion patterns and handles uncertainty in detection.

5. Streamlit Deployment

Streamlit is an open-source Python library for building interactive web applications for data science and machine learning. In our project, it is used to provide a user-friendly interface:

- Upload a video file directly from the UI.
- Run object detection and segmentation pipelines.
- Visualize motion tracking results in real-time.

3 Approach 1: Edge Detection + ORB Feature Tracking

3.1 Pipeline

1. Extract frames from input video.
2. Apply Canny edge detection for motion segmentation.
3. Use ORB to detect and track keypoints across frames.
4. Overlay edge and feature maps on original frames.
5. Compile processed frames into a video.

3.2 Frame Preprocessing and Feature Detection



Figure 1: Converted grayscale frame before applying filters.



Figure 2: Detected edges using Canny edge detector.

3.3 ORB Keypoints Detection



Figure 3: Keypoints detected using ORB.

3.4 Optical Flow for Segmentation

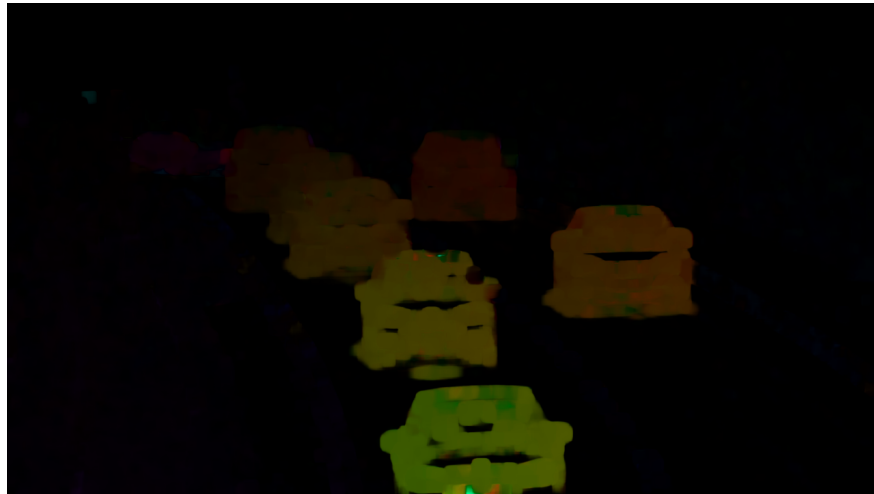


Figure 4: Motion segmentation using Farneback optical flow.

3.5 Lucas-Kanade Feature Tracking



Figure 5: Tracked feature points with Lucas-Kanade method.

3.6 Kalman Filter Output

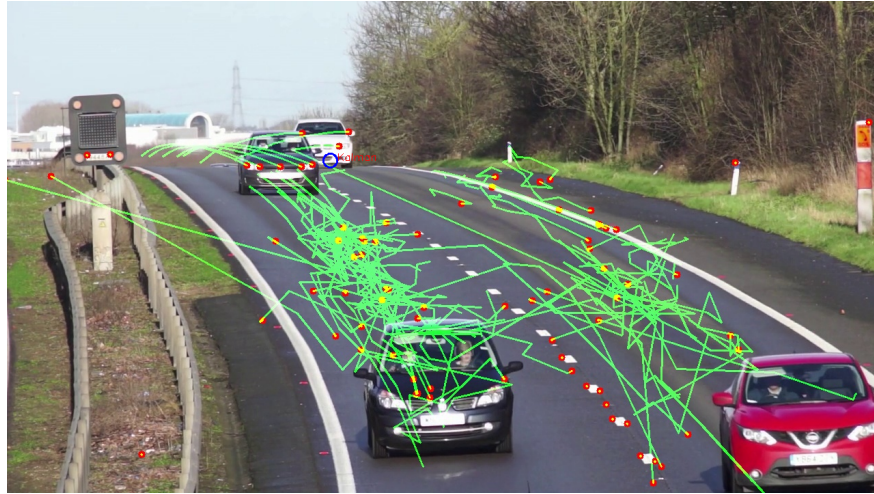


Figure 6: Predicted object path using Kalman filter.

3.7 Homography Transformation

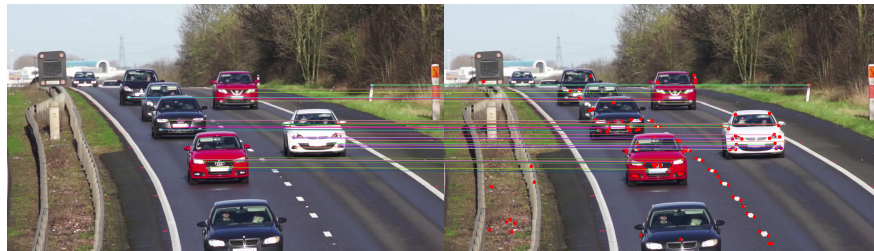


Figure 7: Warped image after computing homography.

Colab link of approach 1: https://colab.research.google.com/drive/1EC60joIneiwkkLEFvV_kN-C0Ttjpwhdb?usp=sharing

4 Approach 2: Frame Differencing + Thresholding

4.1 Pipeline

- Convert video frames to grayscale.
- Compute absolute difference between consecutive frames.
- Apply binary thresholding to isolate motion.

- Highlight motion in green overlay.
- Reconstruct video from selected processed frames.

4.2 Frame Difference Image



Figure 8: Difference image between consecutive grayscale frames.

4.3 Motion Highlight via Thresholding



Figure 9: Motion segmented using binary thresholding.

5 Comparison Table

Aspect	Approach 1 (ORB)	Approach 2 (Diff)
Speed	Medium	Fast
Accuracy	High (keypoints)	Low (sensitive to noise)
Usability	Robust to rotation	Good for simple motion

Table 1: Comparison between the two classical CV approaches used.

6 Implementation Details

Tools Used: OpenCV, NumPy, Streamlit, Google Colab.

Common Steps:

- Video reading and frame extraction using OpenCV.
- Preprocessing (grayscale conversion, resizing).
- Motion analysis and segmentation per approach.
- Final video generation.
- Streamlit-based deployment.

7 Results

- Both methods produce motion-highlighted videos.
- Frame differencing is simpler but less robust.
- ORB tracking offers better tracking of fine movement and corners.

8 Deployment

Deployed using Streamlit for interactive demonstration of motion detection results.

App Link: <https://8nemhnoyev3wi3zpjazcv.streamlit.app>

GitHub Repo: <https://github.com/VanshikaKumawat/video-segmentation-and-tracking->

Demo Videos: https://drive.google.com/file/d/1t2U7Ro4HvX9e3DLwoStzfrHlk_4fXoHq/view?usp=drive_link

9 Screenshots of Output

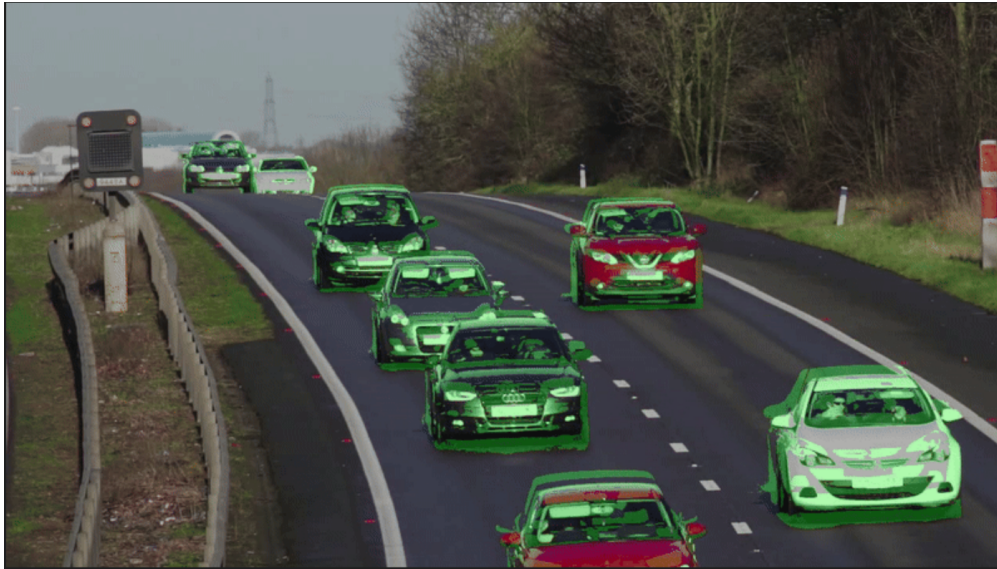


Figure 10: Object detection applied to traffic video.



Figure 11: Motion detection applied to aquatic video.



Figure 12: Motion detection applied to butterfly video.

10 Conclusion

This project demonstrates that classical Computer Vision techniques can effectively detect and track motion in videos. While Approach 1 is more robust to noise and handles complex motion better, Approach 2 is faster and suitable for basic surveillance systems.